CCBDA-MSE1-Important

1. Explain the various classes of data used for data processing with examples.

There are **3 types of data**:

1. Structured Data

- Data that has a **fixed and predefined structure**.
- Usually stored in a **database using rows and columns**.
- Easy to **store, search, update and process**.
- Uses a fixed schema.
- **Examples:** Student records, employee records, customer records.

Easy to remember:
👉 **Structured = Proper table**

Example:

ID	Name	Marks
101	Rahul	85
102	Priya	90

2. Semi-Structured Data

- Does **not have a fixed table structure**.
- But it still has **some organization or structure**.
- Usually uses **tags or metadata**.
- **Examples:** XML, JSON, HTML, emails.

Easy to remember:
👉 **Semi-structured = Some structure**

Example:

```
{
  "name": "Rahul",
  "marks": 85
}
```

3. Unstructured Data

- Does **not have a predefined structure**.

- Cannot easily be stored in rows and columns.
- Difficult to organize and process using traditional databases.
- **Examples:** Images, videos, audio, presentations, letters.

Easy to remember:
👉 **Unstructured = No proper structure**

⭐ Remember:

Structured → Table
Semi-structured → Tags/JSON/XML
Unstructured → Images/Video/Audio

These three categories and their examples are directly covered in the PDF.

---

2. Discuss 3 V's of Big Data and Challenges of Big Data.

3 V's of Big Data

Big Data is mainly described using **3 V's**:

1. Volume → "How much data?"

- Means the **huge amount of data** generated and stored.
- Data comes from databases, sensors, social media, websites, videos, documents, etc.
- Because the amount is huge, we need systems capable of storing and processing it.

Remember:
👉 **Volume = Amount**

2. Velocity → "How fast?"

- Means the **speed at which data is generated, collected and processed**.
- Some data needs to be processed immediately.
- Examples:
    - Stock market data
    - IoT sensor data
    - Online transactions
    - Social media feeds

Processing can move from:

Batch → Periodic → Near Real-time → Real-time

Remember:
👉 **Velocity = Speed**

3. Variety → "What type?"

- Means the **different types and formats of data**.
- It includes:
    - Structured
    - Semi-structured
    - Unstructured

Remember:
👉 **Variety = Different types**

🧠 Super easy memory trick:

VOLUME = How much?
VELOCITY = How fast?
VARIETY = What type?

The PDF defines the three V's in exactly this way.

Challenges of Big Data

Remember these **6 challenges**:

1. **Exponential growth of data**
    - Data keeps increasing very quickly.
    - Difficult to identify useful information from unnecessary data.
2. **Cloud computing and virtualization**
    - Need suitable infrastructure to manage huge data.
    - Organizations must decide how and where to host Big Data systems.
3. **Data retention**
    - Difficult to decide **how long data should be stored**.
    - Some data remains useful for years, while some becomes useless quickly.
4. **Lack of skilled professionals**
    - Need people with skills in data science and Big Data technologies.
5. **Data processing challenges**
    - Difficulties in storing, searching, analyzing, transferring and securing huge amounts of data.
6. **Data visualization**
    - Huge amounts of data are difficult to present in a simple and understandable way.

Easy memory:

Growth → Cloud → Retention → Skills → Processing → Visualization

---

3. "It is easy to work with structured data compared to unstructured data." Justify with an example.

Basic idea

Structured data is easier because it is organized in a fixed format.

For example, consider a college database:

Student ID	Name	Course	Semester	Marks
S101	Rahul	MCA	2	82
S102	Priya	MCA	2	91

Everything is properly organized into **rows and columns**.

So we can easily:

1. Insert, Update and Delete

- Add a new student.
- Change a student's marks.
- Delete a student.

2. Security

- Different users can have different permissions.
- Example:
    - Student → Can view details.
    - Staff → Can modify marks.

3. Indexing

- Index helps us **find data faster**.
- Example: Create an index on the **Course** field to quickly find MCA students.

4. Scalability

- The database can grow from hundreds to thousands of students.
- More storage and processing capacity can be added.

5. ACID Transactions

Structured databases support **ACID** properties:

- **A – Atomicity:** All or nothing.
- **C – Consistency:** Data remains correct.
- **I – Isolation:** Transactions don't interfere with each other.
- **D – Durability:** Saved data remains saved even after a crash.

Why is unstructured data difficult?

Imagine you have:

- 📷 Student photograph
- 🎥 Recorded lecture
- 📧 Email
- 🎤 Voice recording

They don't have a fixed **row-column structure**.

Therefore, finding and processing specific information is more difficult.

⭐ Easy conclusion:

> **Structured data → Organized → Easy to search/process**
> **Unstructured data → Not organized → Difficult to search/process**

This question's example and the five advantages are given in pages 5–6 of your PDF.

---

4. Explain unstructured data. How to deal with it? Explain techniques used to find unstructured data.

What is Unstructured Data?

Unstructured data is data that:

- Does **not have a predefined structure**.
- Does not follow a fixed data model.
- Cannot easily be stored in rows and columns.
- Is difficult to process using traditional databases.

Examples:

- Images
- Videos

- Emails
- Audio
- Documents
- Social media content

Simple example:

A student table:

ID | Name | Marks

is structured.

But a student's:

📹 Video + 📷 Photo + 🎤 Voice recording

is unstructured.

How do we deal with Unstructured Data?

The PDF gives these techniques:

1. Data Mining

Used to find **patterns and relationships** in large datasets.

Important techniques:

- **Association Rule Mining** → Finds what things occur together.
  - Example: People buying bread may also buy eggs.
- **Regression Analysis** → Predicts relationships between variables.
- **Collaborative Filtering** → Predicts what a user may prefer based on other users' preferences.

2. Text Analytics / Text Mining

- Extracts meaningful information from text.
- Finds patterns and trends.
- Includes:
  - Text categorization
  - Text clustering
  - Sentiment analysis
  - Entity extraction

3. Natural Language Processing (NLP)

- Helps computers **understand human language**.

Example:

> "I really loved this movie!"

NLP can help a computer understand that the statement has a **positive sentiment**.

4. Noisy Text Analytics

Deals with messy text such as:

- Chats
- Blogs
- Emails
- Text messages

It handles things like:

- Spelling mistakes
- Abbreviations
- Missing punctuation
- Filler words such as "uh" and "um"

5. Manual Tagging with Metadata

- Manually add **metadata/tags** to unstructured data.
- This helps provide meaning and makes the data easier to understand.

6. POS Tagging

POS = **Part-of-Speech Tagging**

It identifies whether a word is:

- Noun
- Verb
- Adjective
- etc.

7. UIMA

Unstructured Information Management Architecture

- An open-source platform from IBM.
- Used for analyzing text and other unstructured data.
- Helps find hidden meaning and relationships.

🧠 Easy order to remember:

Data Mining → Text Mining → NLP → Noisy Text → Metadata → POS → UIMA

---

5. Difference between Traditional BI and Big Data

This one is important because you asked about it earlier.

Traditional BI	Big Data
Data is stored in a **central server**	Data is stored in a **distributed file system**
Usually **scales vertically** — make the existing server more powerful	**Scales horizontally** — add more computers
Mainly analyzes data in **offline mode**	Analyzes data in **real-time as well as offline mode**
Mainly deals with **structured data**	Deals with **structured + semi-structured + unstructured data**
Data is moved to the processing function	**Processing function is moved to the data**

What does the last point mean?

Traditional BI:

DATA → Processing

You **bring the data to the processing system**.

Big Data:

DATA
↓
Processing is brought here

The **processing function goes to where the data is stored**.

🧠 Super-easy memory:

Remember these 5 differences:

Central → Distributed
Vertical → Horizontal
Offline → Real-time + Offline
Structured → All types
Data to Code → Code to Data

These five differences are directly from Question 5 on page 9 of your PDF.

🔥 Final 5-question revision

If you have very little time before the exam, remember this:

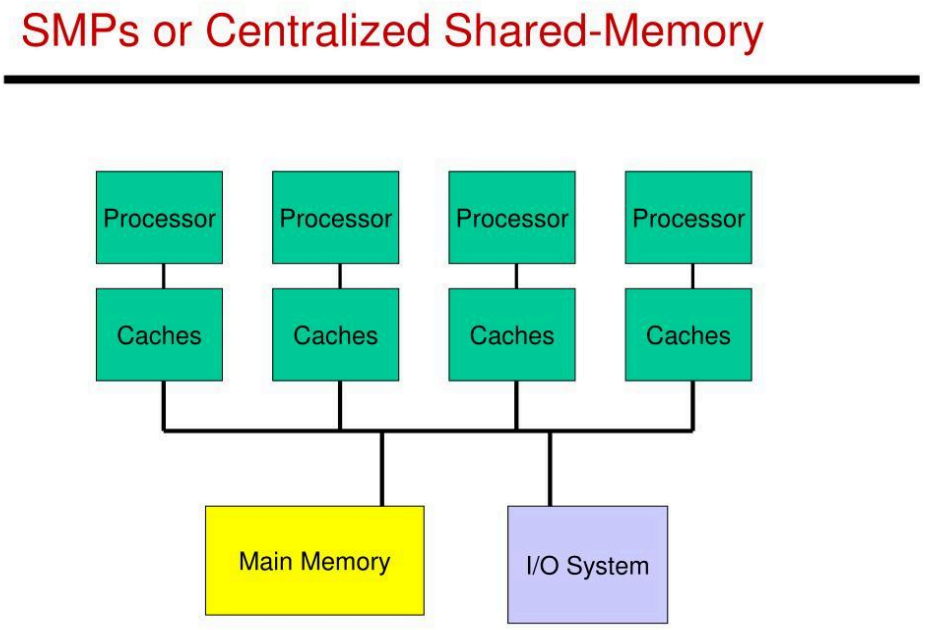
Q	Topic	What to remember
1	Types of Data	**Structured, Semi-structured, Unstructured**
2	3 V's + Challenges	**Volume, Velocity, Variety + 6 challenges**
3	Structured vs Unstructured	**Structured is organized → easier to process**
4	Unstructured Data	**Definition + 7 techniques**
5	Traditional BI vs Big Data	**Central/Distributed, Vertical/Horizontal, Offline/Real-time, Structured/All, Data-to-Code/Code-to-Data**

This way, you don't have to memorize long paragraphs. **Understand the bold idea first, then expand each point in the exam.**

6. Explain the following a)SMP b)Shared nothing architecture

a) SMP – Symmetric Multiprocessing

SMP (Symmetric Multiprocessing) is a computer architecture in which **two or more identical processors share a common main memory and I/O resources**. All processors are controlled by a **single operating system**, and each processor is treated equally.



Working

```
Single Operating System
          |
  ┌───────┼───────┐
  ↓       ↓       ↓
CPU 1   CPU 2   CPU 3
  |       |       |
  └───────┼───────┘
          ↓
    Shared Memory
          |
      I/O Devices
```

All processors can access the **same shared memory and common resources**. Different processors can execute different programs or work on different sets of data simultaneously.

Features of SMP

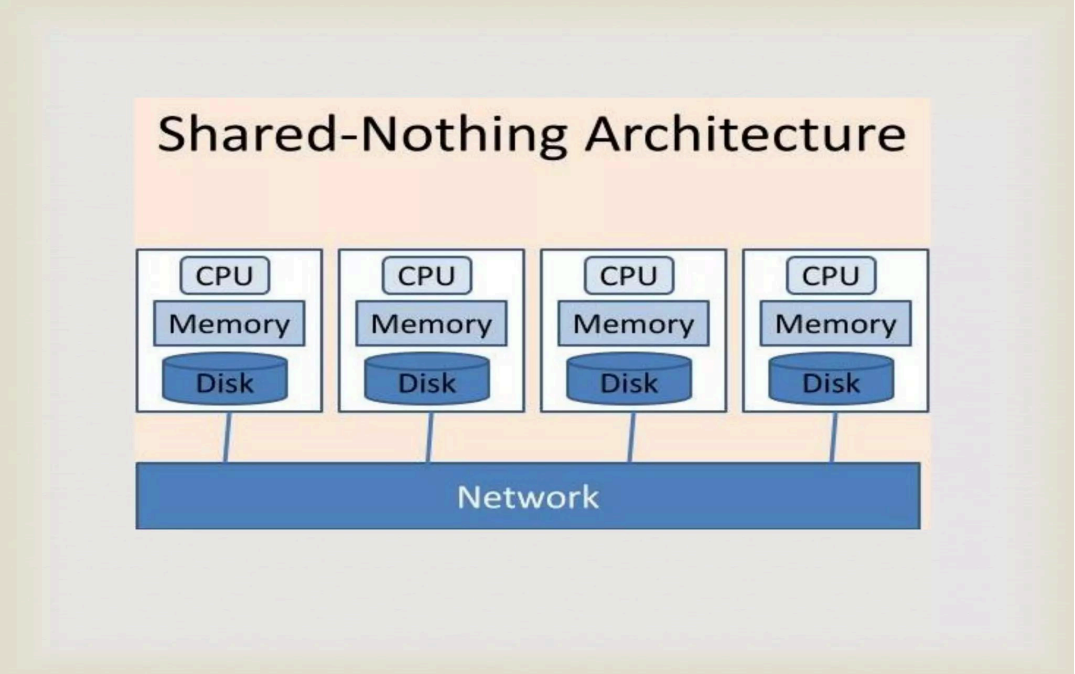
- Uses **multiple processors**.
- Processors share **common main memory**.
- All processors are managed by **one operating system**.
- Processors work independently and can execute tasks simultaneously.
- Provides **parallel processing**, improving performance.

Example: Multi-core computers where multiple CPU cores are treated as separate processors.

---

b) Shared Nothing Architecture

Shared Nothing (SN) architecture is a **distributed computing architecture** in which each node has its own **CPU, memory, and disk/storage**. No memory or disk is shared between the nodes.



Working

```
          Network
      ┌─────┼─────┐
      ↓     ↓     ↓
  Node 1  Node 2  Node 3
    |       |       |
CPU+RAM  CPU+RAM  CPU+RAM
  +Disk    +Disk    +Disk
```

Each node operates **independently** and communicates with other nodes through a network. Since resources are not shared, additional nodes can be added to increase the system's capacity.

Features / Advantages

- Each node has its **own CPU, memory, and storage**.
- No common memory or disk is shared between nodes.
- Supports **horizontal scaling** — more nodes can be added easily.
- Reduces the **single point of failure**.
- If one node fails, other nodes can continue working.
- Suitable for **large distributed databases and Big Data systems**.

SMP vs Shared Nothing

SMP	Shared Nothing
Multiple processors share memory.	Each node has its own memory.
Processors operate under one OS.	Nodes operate independently.
Tightly coupled system.	Distributed/loosely coupled system.
Scaling is generally limited by shared resources.	Easily supports horizontal scaling.
Example: Multi-core computer.	Example: Distributed database cluster.

In short:
SMP = Multiple processors + Shared memory
Shared Nothing = Multiple nodes + No shared memory or disk.

7. Compare parallel and distributed systems and discuss the importance of NOSQL

1. Parallel Systems

A **parallel system** uses multiple processors to execute different parts of a program **simultaneously**. The main objective is to reduce processing time and improve performance.

For example, a large calculation can be divided into smaller tasks and processed by multiple processors at the same time.

The course material describes **SMP** as a tightly coupled multiprocessor system where processors share common memory and resources. **MPP** is another parallel approach where multiple processors work on different parts of a program, each having its own memory and operating system.

2. Distributed Systems

A **distributed system** consists of multiple independent computers or nodes connected through a network. Each node has its own processing, memory, and storage resources, and the nodes communicate with each other to perform a task.

Example: A Big Data system where data is distributed across many servers and processed simultaneously.

Parallel vs Distributed Systems

Parallel System	Distributed System
Uses multiple processors to perform tasks simultaneously.	Uses multiple independent computers/nodes.
Processors may share memory and other resources.	Nodes generally have their own memory and storage.
Usually more tightly coupled.	Usually loosely coupled.
Communication is generally through shared memory or high-speed interconnects.	Communication occurs through a network.
Mainly focuses on increasing processing speed.	Focuses on scalability, availability, and distributed processing.
Example: **SMP/MPP systems**.	Example: **Hadoop cluster**.

In Big Data environments, distributed architectures are important because data can be very large, arrive quickly, and have many different forms, requiring distributed systems rather than traditional architectures.

Importance of NoSQL

NoSQL (Not Only SQL) databases are non-relational databases designed to handle **large-scale, distributed, and diverse data**. They are particularly important in Big Data applications.

1. Handles Large Volumes of Data

NoSQL databases can manage huge amounts of data generated from sources such as transactions, sensors, logs, social media, and multimedia.

2. High Scalability

NoSQL databases support **horizontal scaling**, where additional servers can be added to handle increasing data and workloads.

3. Flexible Schema

Unlike traditional relational databases with rigid schemas, NoSQL databases can support **flexible or dynamic data structures**. This is particularly useful when the structure of data changes frequently.

4. Handles Different Types of Data

NoSQL can handle **structured, semi-structured, and unstructured data**, which is important because Big Data contains many different formats.

5. High Availability

NoSQL databases can distribute and replicate data across multiple nodes, allowing applications to continue operating even when some nodes fail.

6. Suitable for Distributed Applications

NoSQL databases are designed to work effectively in **distributed environments**, making them suitable for cloud computing and Big Data applications.

Examples

- **MongoDB** – Document database
- **Redis** – Key-value database
- **Cassandra** – Wide-column database
- **Neo4j** – Graph database

Conclusion

Parallel systems mainly improve performance by processing multiple tasks simultaneously, while **distributed systems** use multiple independent networked computers to process and store data. NoSQL is important in Big Data because it provides **scalability, flexibility, high availability, and the ability to handle large and diverse datasets**.

8. List and explain key terminologies used in big data environments

A **Big Data environment** consists of technologies and concepts used to **store, process, manage, and analyse very large and complex datasets**. Some important terminologies are given below.

1. Big Data

Big Data refers to datasets that are extremely large, complex, and generated at high speed, making them difficult to handle using traditional data-processing systems.

Example: Social media data, sensor data, transaction records, and web logs.

2. Hadoop

Hadoop is an open-source framework designed for the **storage and processing of Big Data**.

It can distribute data and processing across multiple computers.

3. Hadoop Cluster

A **Hadoop cluster** is a group of connected computers or **nodes** that work together to store and process Big Data.

A node can use relatively inexpensive hardware, such as a processor, network card, and hard drives.

4. HDFS

HDFS (Hadoop Distributed File System) is the storage system of Hadoop. It stores large files by **dividing them into blocks and distributing them across different nodes**.

This allows large datasets to be stored and processed efficiently.

5. MapReduce

MapReduce is a distributed processing model used to process large datasets across multiple nodes.

It mainly consists of two stages:

```
Input Data
    ↓
   Map
    ↓
Intermediate Data
    ↓
  Reduce
    ↓
Output
```

6. NoSQL

NoSQL (Not Only SQL) refers to non-relational databases designed to handle large-scale and distributed data.

Examples include:

- **MongoDB** – Document database
- **Redis** – Key-value database

- **Cassandra** – Wide-column database
- **Neo4j** – Graph database

7. Data Warehouse

A **Data Warehouse** is a centralized repository used to store integrated and historical data for **reporting, analysis, and decision-making**.

8. Data Lake

A **Data Lake** is a centralized repository that stores large amounts of **structured, semi-structured, and unstructured data**, generally in its raw form.

Example: Storing raw sensor data, social-media data, images, videos, JSON files, and database records.

9. Metadata

Metadata means "data about data." It describes information about the structure, meaning, source, ownership, or history of data.

Example: The author, date, and size of a document are metadata about that document.

10. Data Mart

A **Data Mart** is a smaller, subject-specific part of a data warehouse designed for a particular department or business function.

Example: Sales Data Mart, Finance Data Mart, or HR Data Mart.

11. Sharding

Sharding is the process of dividing a large dataset into smaller parts called **shards** and distributing them across multiple servers.

It helps provide **horizontal scalability**.

12. Replication

Replication means maintaining multiple copies of data on different servers or nodes. It improves **availability and fault tolerance**.

Quick Revision

Terminology	Meaning

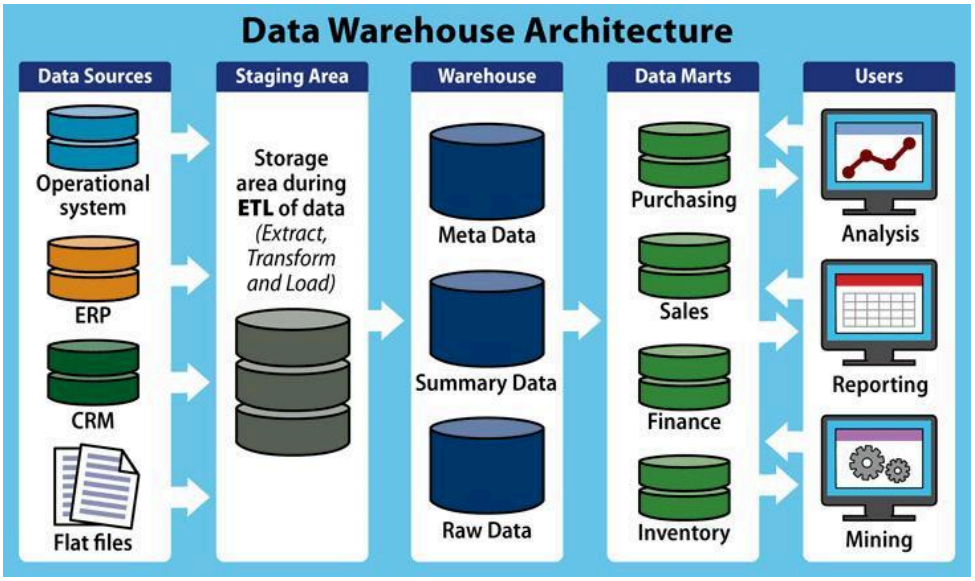
Big Data	Very large and complex datasets
Hadoop	Framework for storing and processing Big Data
Cluster	Group of connected computers/nodes
HDFS	Distributed storage system
MapReduce	Distributed data-processing model
NoSQL	Non-relational database systems
Data Warehouse	Repository for analytical/historical data
Data Lake	Repository for raw structured and unstructured data
Metadata	Data about data
Data Mart	Department-specific subset of a data warehouse
Sharding	Dividing data across servers
Replication	Maintaining copies of data

Conclusion

These terminologies form the basic foundation of a **Big Data environment**. Technologies such as **Hadoop, HDFS, MapReduce, and NoSQL** help organizations store and process huge datasets, while **data warehouses, data lakes, and data marts** help organize data for analysis and decision-making.

9. Explain a typical data warehouse environment and its key components.

A **Data Warehouse environment** is a system that collects data from different sources, cleans and transforms it, stores it in a central warehouse, and provides tools for **reporting, analysis, and decision-making**.



Typical Data Warehouse Environment

```
Operational / External Sources
            |
            ↓
   ┌──────────────────┐
   |   ETL Process    |
   | Extract          |
   | Transform        |
   | Load             |
   └──────────────────┘
            |
            ↓
   ┌──────────────────┐
   | Staging Area     |
   └──────────────────┘
            |
            ↓
  ┌──────────────────────┐
  |   Data Warehouse     |
  |      / EDW           |
  └──────────────────────┘
            |
            ↓
       OLAP Server
            |
            ↓
  ┌─────────┼──────────┐
  ↓         ↓          ↓
Reports   Analysis   Data Mining
```

Key Components

1. Data Sources

These are the systems from which data is collected.

Examples include:

- Relational databases
- Flat files
- XML/JSON files
- Spreadsheets
- Legacy applications
- Transaction systems

A data warehouse can integrate data from different databases, hardware, operating systems, and formats.

2. ETL Process

ETL stands for Extract, Transform, and Load.

- **Extract:** Data is collected from different source systems.
- **Transform:** Raw data is cleaned, validated, mapped, and converted into a suitable format.
- **Load:** The transformed data is loaded into the data warehouse.

3. Staging Area

The **staging area** is a temporary area where extracted data is stored before loading it into the warehouse.

It allows data to be **validated, cleaned, and transformed** without affecting the source systems.

4. Data Warehouse

The **Data Warehouse** is the central repository where integrated and historical data is stored for analysis and decision-making.

It is generally **subject-oriented, integrated, time-variant, and non-volatile**.

5. OLAP Server

OLAP (Online Analytical Processing) provides fast analysis of multidimensional data.

Two common types are:

- **ROLAP:** Relational OLAP
- **MOLAP:** Multidimensional OLAP

6. Data Marts

A **Data Mart** is a smaller portion of the data warehouse designed for a specific department or business function.

Examples:

- Sales Data Mart
- Finance Data Mart
- HR Data Mart

In a top-down approach, data marts are built on top of the data warehouse.

7. Metadata Repository

Metadata means information about the data. The metadata repository stores information about the warehouse structure, schemas, dimensions, hierarchies, data marts, data sources, transformation rules, and operational information.

8. Front-End / Client Tools

The top layer provides tools through which users access and analyse warehouse data.

It includes:

- Query and reporting tools
- Data analysis tools
- Data mining tools
- Visualization tools

These tools help users generate reports, identify trends, and perform analysis.

Conclusion

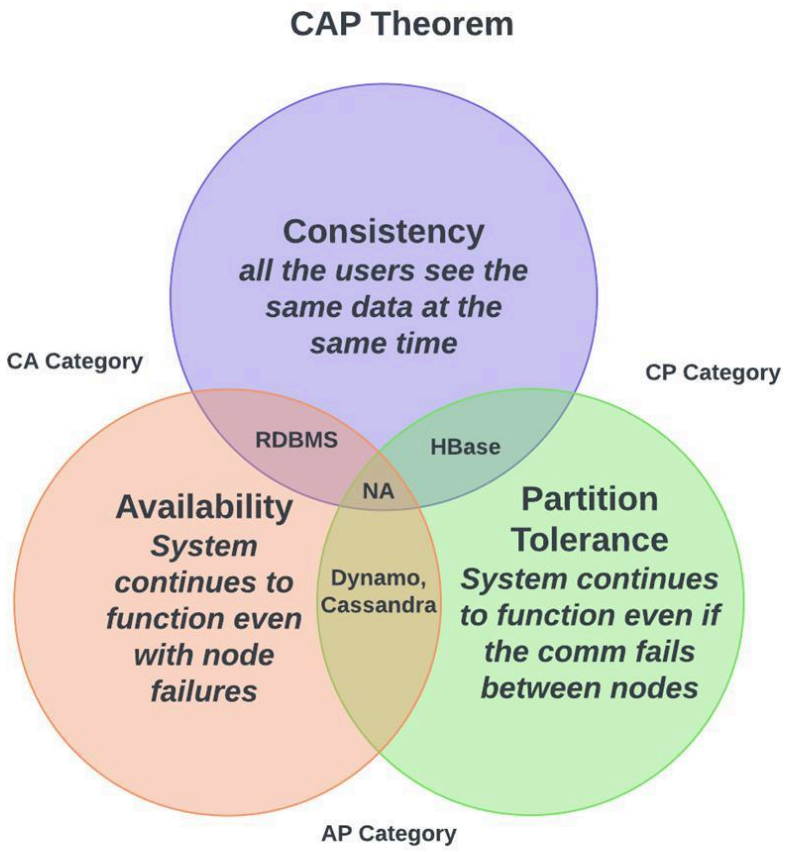
A typical data warehouse environment follows the flow:

Data Sources → ETL → Staging Area → Data Warehouse → OLAP → Reports/Analysis

The major components work together to convert **raw data into meaningful information** that supports business analysis and decision-making.

10. Explain CAP theorem and its significance in distributed systems.

CAP Theorem, also known as **Brewer's Theorem**, is a fundamental principle in distributed computing. It states that a distributed system cannot simultaneously guarantee all three of the following properties. At most, **two out of the three** can be guaranteed.



Three Properties of CAP Theorem

1. Consistency (C)

Consistency means that every client gets the **same and most recent data** after an operation is completed.

For example, if a user's account balance is updated, all clients accessing the database should see the updated balance.

2. Availability (A)

Availability means that the system remains **operational and responds to requests**, even when some components fail.

In simple terms, the service should remain available without downtime.

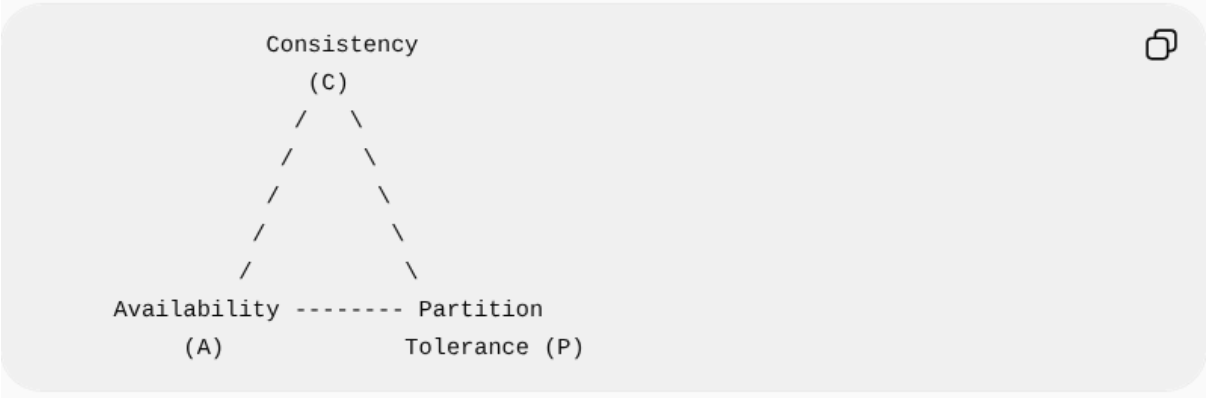
3. Partition Tolerance (P)

Partition Tolerance means that the system continues to operate even when **communication between some servers is lost** or the network is partitioned.

The servers may be divided into groups that temporarily cannot communicate with each other.

CAP Trade-off

The important idea is:



A distributed system can choose combinations such as:

CA – Consistency + Availability

The system provides both consistency and availability **when there is no network partition**. If a partition occurs, the system may block operations rather than continue with inconsistent data.

CP – Consistency + Partition Tolerance

The system maintains **consistent and accurate data** even during a partition, but some data may become temporarily unavailable.

Example: A banking system may prefer consistency over availability because incorrect account information is unacceptable.

AP – Availability + Partition Tolerance

The system continues to remain **available during a partition**, but some returned data may temporarily be inconsistent or outdated.

Example: A social-media application may allow a post or update to appear on different servers at slightly different times.

Significance of CAP Theorem

1. **Helps design distributed systems:** It helps developers understand the trade-offs involved when designing distributed databases.

2. **Helps choose databases:** Different NoSQL databases make different choices regarding consistency and availability.
3. **Important for NoSQL:** CAP is particularly relevant to distributed NoSQL databases because they commonly distribute data across multiple nodes.
4. **Helps handle network failures:** Partition tolerance allows systems to continue operating despite communication failures between nodes.
5. **Supports business requirements:** The choice between consistency and availability depends on the application's requirements. For example, systems requiring accurate transactions may prioritize consistency, while highly available applications may accept temporary inconsistency.

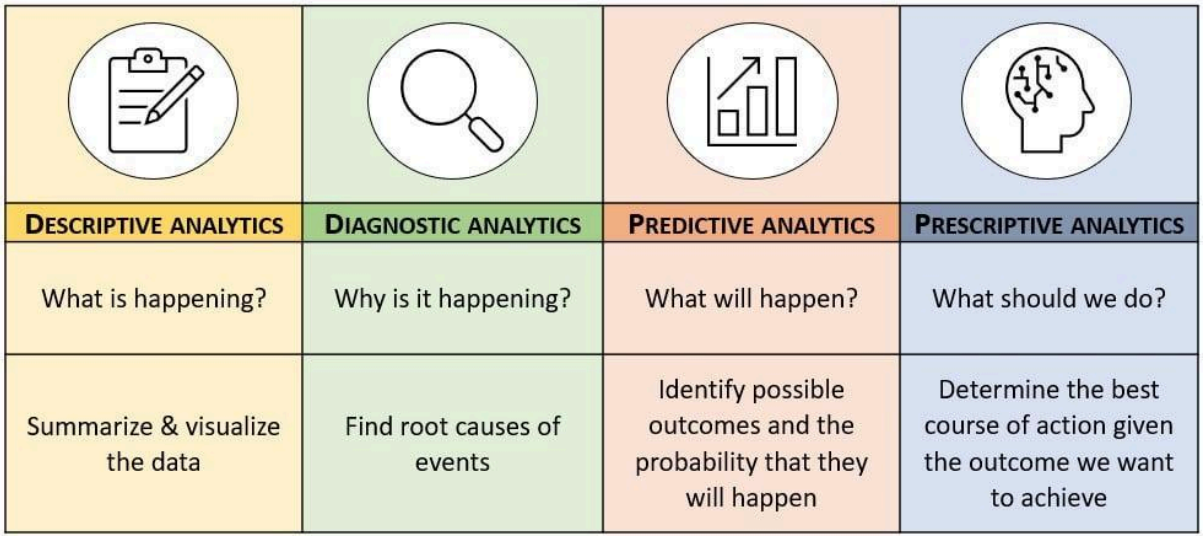
Conclusion

The **CAP theorem states that a distributed system cannot guarantee Consistency, Availability, and Partition Tolerance simultaneously**. Therefore, system designers must make appropriate trade-offs depending on the application's requirements. It is especially important when designing **distributed systems and NoSQL databases**.

11. Explain how Big Data Analytics can be categorized. Discuss the different categories with suitable examples.

Big Data Analytics is the process of examining large and varied datasets to discover **patterns, correlations, trends, and useful insights** that help in better decision-making.

Big Data Analytics can be categorized based on the **type of question being answered and the type of insight produced**.



1. Descriptive Analytics

Descriptive Analytics answers:

> **“What happened?”**

It analyses **past and historical data** and summarizes events or occurrences that have already happened.

Examples:

- Monthly sales report
- Number of customers who purchased a product
- Previous year's company revenue
- Website traffic report

The course material describes Analytics 1.0 primarily as descriptive statistics used to report past events.

---

2. Diagnostic Analytics

Diagnostic Analytics answers:

> **“Why did it happen?”**

It analyses historical data to identify the **causes or reasons behind an event or result**.

Example:

If a company's sales decreased in June, diagnostic analytics can analyse customer data, pricing, competitors, and regional sales to determine **why sales decreased**.

The material identifies questions such as **“Why did it happen?”** as part of analytics-based investigation.

---

3. Predictive Analytics

Predictive Analytics answers:

> **“What is likely to happen next?”**

It uses **historical data, statistical methods, and analytical models** to predict future events.

Examples:

- Predicting future sales
- Predicting customer demand
- Predicting stock trends
- Predicting which customers may leave a service

The course material describes predictive analytics as using past data to **make predictions for the future**.

---

4. Prescriptive Analytics

Prescriptive Analytics answers:

> **"What action should be taken?"**

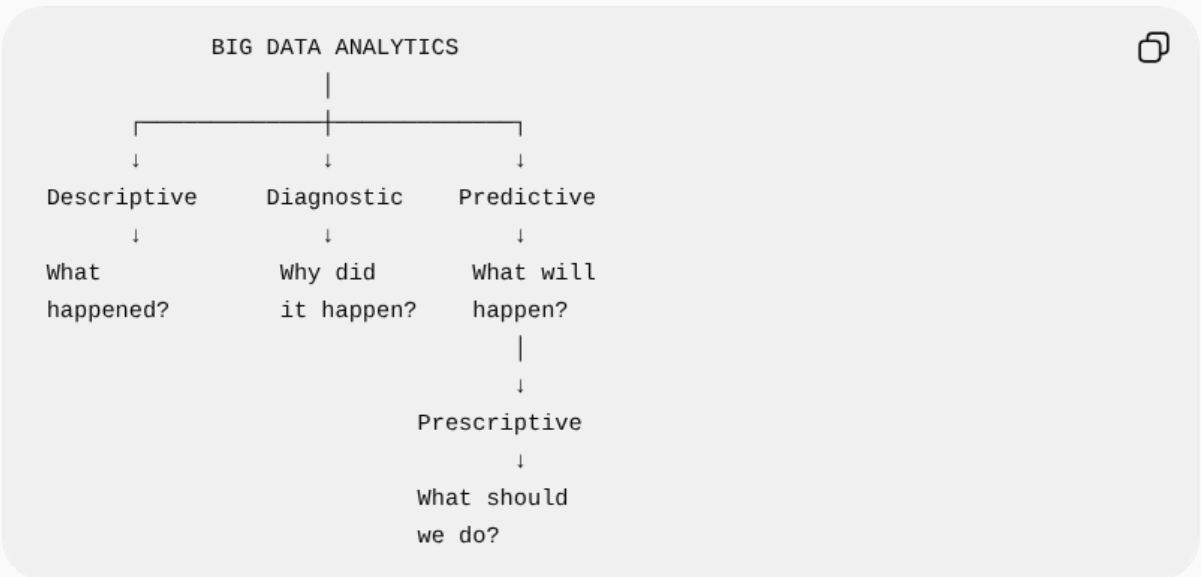
It goes one step beyond prediction. It uses analytical results to **recommend suitable actions** to take advantage of a predicted situation.

Examples:

- Recommending how much inventory a company should maintain
- Suggesting the best price for a product
- Recommending which customers should receive an offer
- Suggesting the best route for delivery

The course material describes prescriptive analytics as using predictions and **making recommendations to leverage the situation**.

Simple Diagram



Comparison

Category	Main Question	Purpose	Example
Descriptive	What happened?	Understand past events	Sales report
Diagnostic	Why did it happen?	Find causes	Reason for sales decline
Predictive	What will happen?	Forecast future events	Future sales prediction
Prescriptive	What should we do?	Recommend actions	Best pricing strategy

Conclusion

Thus, Big Data Analytics progresses from **understanding the past to taking action for the future**:

Descriptive → Diagnostic → Predictive → Prescriptive

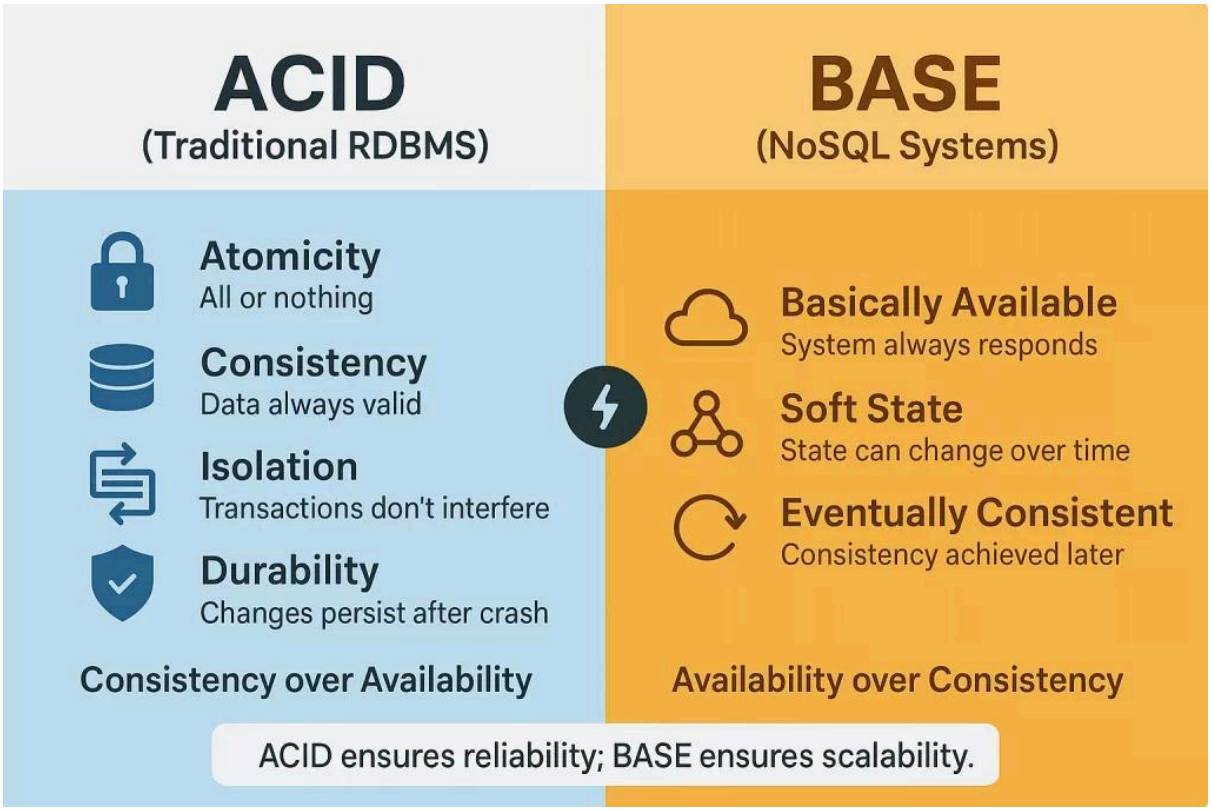
This allows organizations to convert large amounts of data into **useful insights and better decisions**.

12. Explain the concept of BASE in NoSQL databases. Discuss its key features.

BASE is a set of properties used mainly in **distributed and NoSQL databases**. Unlike **ACID**, which gives priority to strong consistency, BASE gives more importance to **high availability, scalability, and speed**.

BASE stands for:

B – Basically Available
S – Soft State
E – Eventually Consistent



1. Basically Available

The system **remains available even when some parts of the distributed system fail**.

In a distributed database, data is stored across multiple sites or servers. If one or two servers go down, the entire system does not necessarily become unavailable. Some data may temporarily become unavailable, but the system continues to operate.

Example:
In a social media application, even if one server fails, users can still access the service through other servers.

---

2. Soft State

Soft state means that the **state of the data can change over time**, even without a new user operation.

BASE allows **temporary inconsistency** between different replicas of the data. This provides flexibility in distributed systems.

Example:
A social media post may appear immediately on one server but take a few seconds to appear on another server.

---

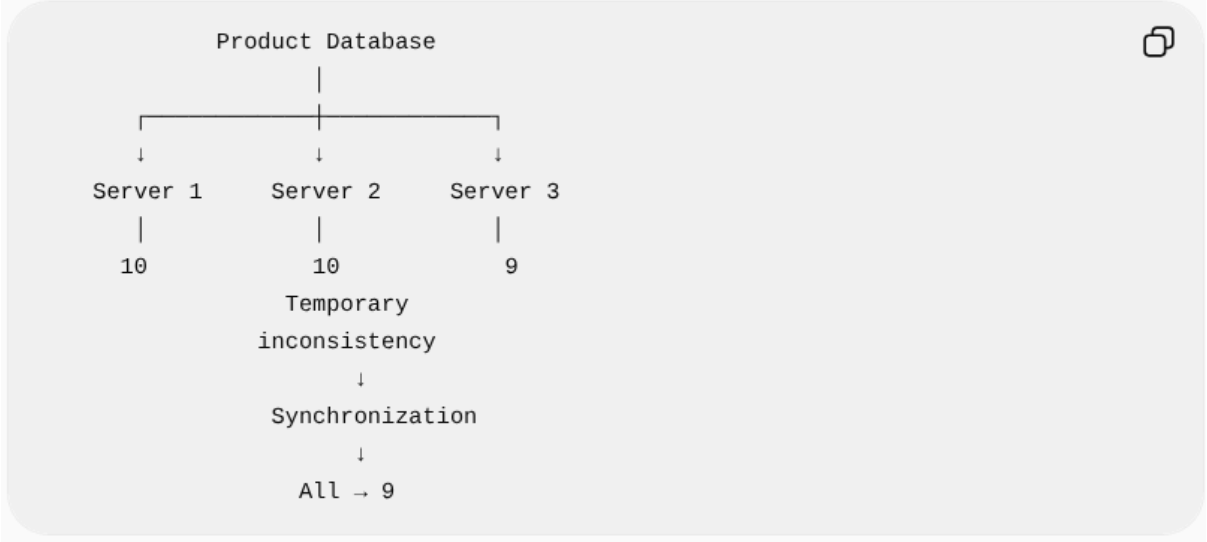
3. Eventually Consistent

Eventually consistent means that **all copies of the data will become consistent after some time**, but consistency is **not guaranteed immediately** after an update.

Example:
Suppose a product's stock changes from 10 to 9. One server may temporarily show **10**, while another shows **9**. After synchronization, all servers will eventually show **9**.

Example of BASE

Consider an online shopping website:



The course material gives a similar example where product availability may temporarily be outdated because of synchronization delays.

Key Features of BASE

Property	Meaning
Basically Available	System remains available even if some components fail
Soft State	Data may temporarily be inconsistent and can change over time
Eventually Consistent	Data becomes consistent after some time

High Availability	Service continues to operate despite partial failures
Scalability	Suitable for large distributed applications
High Performance	Sacrifices immediate consistency for speed and availability

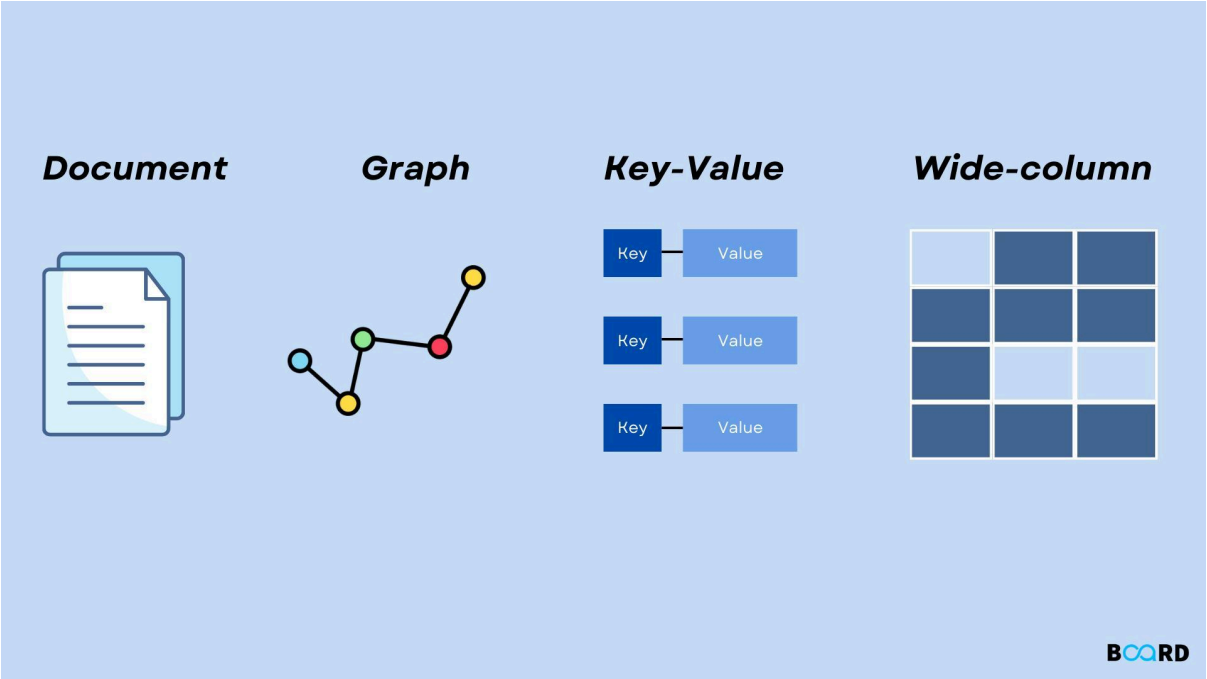
Conclusion

BASE is particularly suitable for **large-scale distributed applications** such as social media and other cloud-based systems, where **availability and speed are often more important than immediate consistency**.

Remember:
BASE = Basically Available + Soft State + Eventually Consistent.

13. Explain the different types of NoSQL databases with suitable examples.

NoSQL (Not Only SQL) databases are non-relational databases designed to store and manage data in flexible formats rather than traditional tables with rows and columns. The four main types are **Document, Key-Value, Wide-Column, and Graph databases**.



1. Document-Oriented Database

A **document database** stores data in the form of documents, usually using JSON-like structures. Each document can have different fields, making the database flexible and suitable for changing data.

Example: MongoDB

```json
{
  "name": "Akash",
  "age": 24,
  "skills": ["Java", "React"]
}
```

MongoDB stores documents inside **collections** and does not require every document to have exactly the same fields.

Applications: Content management, e-commerce, web applications.

---

2. Key-Value Database

A **key-value database** stores data as a collection of **key-value pairs**.

```
Key             Value
--------------------------------
user101         "Akash"
user102         "Rahul"
```

The **key** uniquely identifies the value, allowing very fast data retrieval.

Example: Redis.

Applications: Caching, session management, real-time applications.

---

3. Wide-Column Database

A **wide-column database** stores data in **rows and flexible columns**, where different rows can have different sets of columns. It is designed to handle very large amounts of distributed data.

Example: Apache Cassandra.

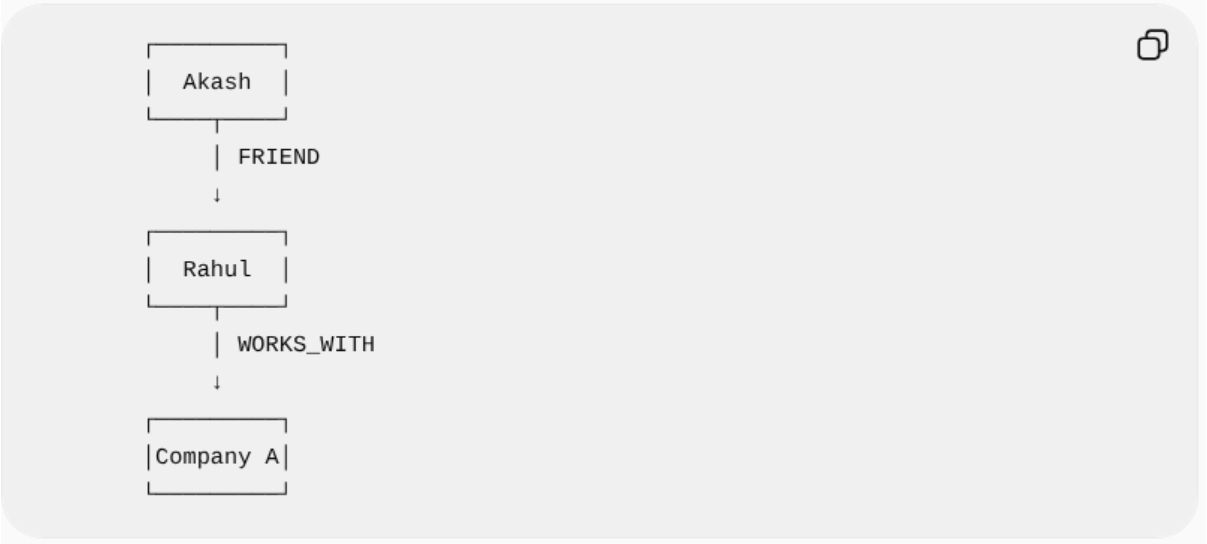
Applications: Large-scale data storage, IoT systems, analytics, and applications requiring high scalability.

---

4. Graph Database

A **graph database** represents data using **nodes and relationships (edges)**. It is especially useful when the relationships between data items are important.

For example:



Example: Neo4j.

Applications: Social networks, recommendation systems, fraud detection, and relationship analysis.

Comparison

Type	Data Structure	Example	Common Use
Document	JSON-like documents	**MongoDB**	Web applications
Key-Value	Key → Value	**Redis**	Caching
Wide-Column	Rows with flexible columns	**Cassandra**	Big-data applications
Graph	Nodes + Relationships	**Neo4j**	Social networks

Conclusion

The four major types of NoSQL databases are **Document, Key-Value, Wide-Column, and Graph databases**. Each type is designed for a particular data structure and workload. **MongoDB**, for example, is a **document-oriented NoSQL database** that stores data as flexible JSON-like documents.

14. Write short notes on the following:
a) Sharding
b) Replication
c) Data Mart

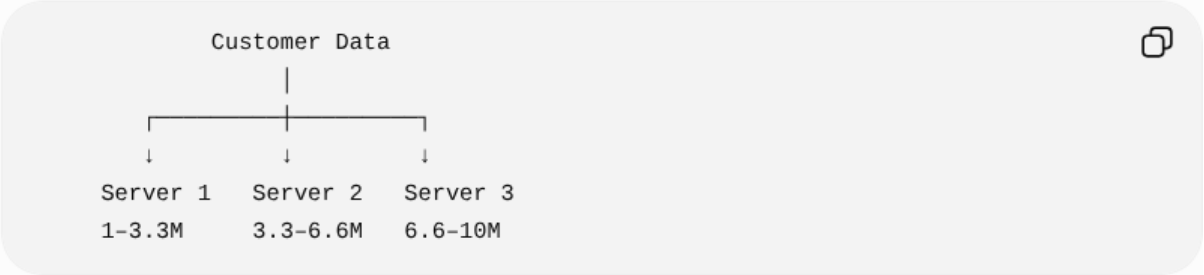
a) Sharding

Sharding is a technique used in distributed databases to **divide a large dataset into smaller parts called shards** and distribute them across multiple servers.

Each server stores a portion of the data. This allows the system to handle large amounts of data and supports **horizontal scaling**. NoSQL databases such as MongoDB can distribute data across multiple servers.

Example

Suppose an e-commerce application has **10 million customer records**:



Instead of storing all data on one server, the data is divided among several servers.

Advantages:

- Handles huge datasets
- Improves performance
- Supports horizontal scaling
- Reduces workload on a single server

---

b) Replication

Replication is the process of maintaining **multiple copies of the same data on different servers or nodes**.

The main purpose is to improve **availability, reliability, and fault tolerance**. If one server fails, another replica can continue serving the data.



For example, in an online shopping system, product information may be replicated across multiple servers. If one server becomes unavailable, another server can continue providing the information.

Replication is closely related to distributed-system availability and fault tolerance.

Advantages:

- High availability
- Fault tolerance
- Better reliability
- Can improve read performance

---

c) Data Mart

A **Data Mart** is a **subset of a data warehouse** designed to serve the needs of a **specific department or group of users**.

For example, an organization may have separate data marts for:



A **Sales Data Mart** may contain only customer, product, and sales-related information. The data in a data mart is generally more focused and summarized.

Advantages:

- Focused on a particular department
- Faster analysis because it contains a smaller amount of data
- Easier to manage
- Helps departments make better decisions

Quick Difference

Concept	Main Purpose
Sharding	Divides data across multiple servers
Replication	Creates copies of data across multiple servers
Data Mart	Stores a department-specific subset of a data warehouse

15. Differentiate between SQL and NoSQL databases.

SQL (Structured Query Language) databases are relational databases that store data in structured **tables with rows and columns**. **NoSQL (Not Only SQL) databases** are non-relational databases that store data in flexible structures such as documents, key-value pairs, wide columns, or graphs.



Key Differences

SQL Database	NoSQL Database
Relational database.	Non-relational database.
Data is stored in **tables, rows, and columns**.	Data can be stored as **documents, key-value pairs, wide columns, or graphs**.

Usually follows a **fixed/predefined schema**.	Generally provides a **flexible schema**.
Uses **SQL** for querying and managing data.	Uses database-specific query methods and APIs.
Relationships between tables are commonly maintained using **primary keys and foreign keys**.	Relationships may be represented through embedded documents or other structures.
Supports **normalization** to reduce data duplication.	Often uses **denormalization**, which may result in duplicated data.
Scaling is traditionally **vertical** by increasing the capacity of a server.	Designed to support **horizontal scaling** by distributing data across multiple servers.
Suitable when strong relationships and consistency are important.	Suitable for large-scale, flexible, and distributed applications.
Examples: **MySQL, PostgreSQL, Oracle, SQLite**.	Examples: **MongoDB, Redis, Cassandra, Neo4j**.

Example

SQL:

```
Student Table
-------------------------
ID | Name  | Department
1  | Akash | MCA
2  | Rahul | MBA
```

NoSQL (MongoDB):

```json
{
  "name": "Akash",
  "department": "MCA",
  "skills": ["Java", "React"]
}
```

MongoDB stores data as **JSON-like documents**, and documents in the same collection can have different fields.

Conclusion

SQL databases are preferred for **structured data, relationships, and strong consistency**, while NoSQL databases are preferred when applications require **flexible data structures, horizontal scalability, and handling of large-scale distributed data**.

Extra Questions

1.Explain why NoSQL databases are used. Discuss the major advantages of NoSQL databases.

NoSQL (Not Only SQL) databases are **non-relational databases** used to store and manage large amounts of data without requiring the fixed table-and-column structure of traditional SQL databases.

They are especially useful in **Big Data and distributed applications**, where data is large, diverse, rapidly changing, and generated from many sources.

Why are NoSQL Databases Used?

1. To Handle Big Data

Modern applications generate huge amounts of data from **social media, sensors, websites, mobile applications, transactions, and IoT devices**. NoSQL databases can handle such large datasets efficiently.

2. To Handle Different Types of Data

NoSQL can store **structured, semi-structured, and unstructured data**.

For example:

Structured → Tables
Semi-structured → JSON / XML
Unstructured → Images / Videos / Logs

3. To Support Changing Data

In traditional SQL databases, changing the structure of a table may require schema modifications. NoSQL databases generally provide a **flexible schema**, making it easier to accommodate changing requirements.

4. To Support Distributed Systems

NoSQL databases are designed to work across **multiple servers or nodes**, making them suitable for large-scale distributed applications.

5. To Support Large Number of Users

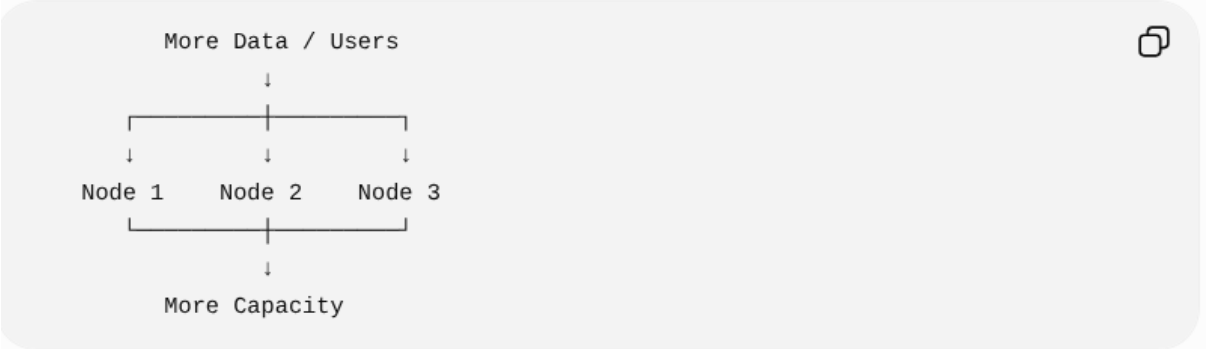
Applications such as social networks, e-commerce websites, and online services may have millions of users. NoSQL databases can scale to support increasing workloads.

Major Advantages of NoSQL Databases

1. High Scalability

One of the major advantages is **horizontal scalability**.

Instead of continuously upgrading one powerful server, additional servers can be added to the system.



2. Flexible Schema

NoSQL databases generally allow data to have a **flexible structure**.

For example, two MongoDB documents can have different fields:

JSON

```json
{
  "name": "Akash",
  "age": 22
}
```

```json
{
  "name": "Rahul",
  "age": 23,
  "skills": ["Java", "React"]
}
```

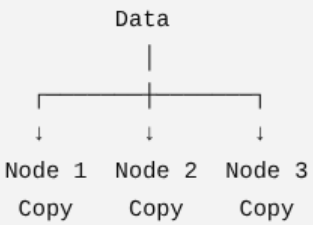
This makes NoSQL suitable when the data structure changes frequently.

3. High Performance

NoSQL databases can provide high performance for workloads suited to their data model. They can efficiently handle large numbers of read and write operations.

4. High Availability

NoSQL systems commonly use **replication**, where copies of data are maintained on multiple nodes.



If one node fails, another node may continue serving the data.

5. Fault Tolerance

Because data and processing can be distributed across multiple nodes, failure of one node does not necessarily bring down the entire system.

6. Easy Horizontal Expansion

New servers can be added as the amount of data or number of users increases. This makes NoSQL particularly suitable for **cloud and distributed environments**.

7. Supports Multiple Data Models

NoSQL databases support different models depending on the application's requirements:

NoSQL Type	Example

Document	MongoDB
Key-Value	Redis
Wide-Column	Cassandra
Graph	Neo4j

8. Suitable for Real-Time and Big Data Applications

NoSQL databases are useful for applications such as:

- Social media
- E-commerce
- IoT
- Real-time analytics
- Online gaming
- Recommendation systems

SQL vs NoSQL — Why NoSQL?

SQL	NoSQL
Fixed/structured schema	Flexible schema
Mainly table-based	Document, key-value, graph, etc.
Traditionally scales vertically	Designed for horizontal scaling
Best suited for highly structured relational data	Suitable for large and diverse data
Relationships are central	Relationships depend on the data model

Conclusion

NoSQL databases are used because modern applications need to handle **large volumes of diverse and rapidly changing data** in distributed environments. Their major advantages are **flexible schema, horizontal scalability, high performance, high availability, fault tolerance, and support for different data models**. Hence, NoSQL is an important technology for **Big Data and modern distributed applications**.

2. Discuss the Application of Big Data Across Various Industries.

Big Data refers to extremely large and complex datasets that are generated from various sources such as social media, sensors, transactions, websites, and mobile applications. Big Data Analytics helps organizations **discover patterns, make predictions, improve operations, and support decision-making**.

1. Healthcare

Big Data is used to analyse **patient records, medical images, laboratory results, and sensor data**.

Applications:

- Predicting diseases
- Personalised treatment
- Patient monitoring
- Analysing medical records
- Improving hospital operations

Example: Patient data can be analysed to identify people who are at high risk of developing a particular disease.

---

2. Banking and Finance

Banks generate huge amounts of transaction and customer data. Big Data helps analyse this information for better financial services.

Applications:

- Fraud detection
- Risk analysis
- Credit scoring
- Customer behaviour analysis
- Real-time transaction monitoring

Example: A bank can identify an unusual transaction pattern and flag it as potentially fraudulent.

---

3. Retail and E-Commerce

Retail companies analyse **customer purchases, browsing history, reviews, and preferences**.

Applications:

- Product recommendations
- Customer segmentation
- Demand forecasting
- Price optimisation
- Inventory management

Example: An e-commerce website recommends products based on a customer's previous purchases and browsing behaviour.

---

4. Transportation

Big Data is generated from **GPS devices, vehicles, traffic sensors, and transportation systems**.

Applications:

- Traffic prediction
- Route optimisation
- Fleet management
- Predictive maintenance
- Improving public transportation

Example: Traffic data can be analysed to identify congested roads and suggest alternative routes.

---

5. Manufacturing

Manufacturing industries use data from **machines, sensors, production lines, and quality-control systems**.

Applications:

- Predictive maintenance
- Quality control
- Production optimisation
- Supply-chain management
- Reducing machine downtime

Example: Sensor data from a machine can be analysed to predict a possible failure before it occurs.

---

6. Telecommunications

Telecommunication companies generate huge amounts of data from **calls, messages, network usage, and customer interactions**.

Applications:

- Network optimisation
- Customer churn prediction
- Service personalisation
- Fraud detection
- Network fault detection

Example: A telecom company can identify customers who are likely to switch providers and offer them suitable plans.

---

7. Education

Educational institutions can analyse data from **student records, examinations, attendance, and online learning platforms**.

Applications:

- Student performance analysis
- Identifying students at risk of dropping out
- Personalised learning
- Course improvement
- Attendance analysis

Example: Student performance data can be analysed to identify students who need additional academic support.

---

8. Agriculture

Big Data is increasingly used in **smart and precision agriculture**.

Data from sensors, satellites, weather systems, and farm equipment can be analysed to improve farming.

Applications:

- Crop monitoring
- Weather prediction
- Soil analysis
- Irrigation management
- Pest and disease detection

Example: Soil-moisture data can help determine when crops need irrigation.

---

9. Government and Public Services

Governments generate and collect large amounts of data related to **citizens, transportation, healthcare, taxation, and public services**.

Applications:

- Public-service planning
- Crime analysis
- Traffic management
- Disaster management
- Resource allocation

Example: Population and transportation data can be used to plan new roads and public facilities.

---

10. Social Media and Entertainment

Social-media and entertainment platforms collect data about **user behaviour, preferences, searches, views, and interactions**.

Applications:

- Content recommendation
- Sentiment analysis
- Targeted advertising
- Trend identification
- Personalised content

Example: A streaming platform analyses viewing history to recommend movies or shows to users.

Summary

Industry	Major Applications
Healthcare	Disease prediction, personalised treatment
Banking & Finance	Fraud detection, risk analysis
Retail	Recommendations, demand forecasting

Transportation	Traffic prediction, route optimisation
Manufacturing	Predictive maintenance, quality control
Telecommunications	Churn prediction, network optimisation
Education	Performance analysis, personalised learning
Agriculture	Crop monitoring, smart irrigation
Government	Public planning, disaster management
Social Media & Entertainment	Recommendations, sentiment analysis

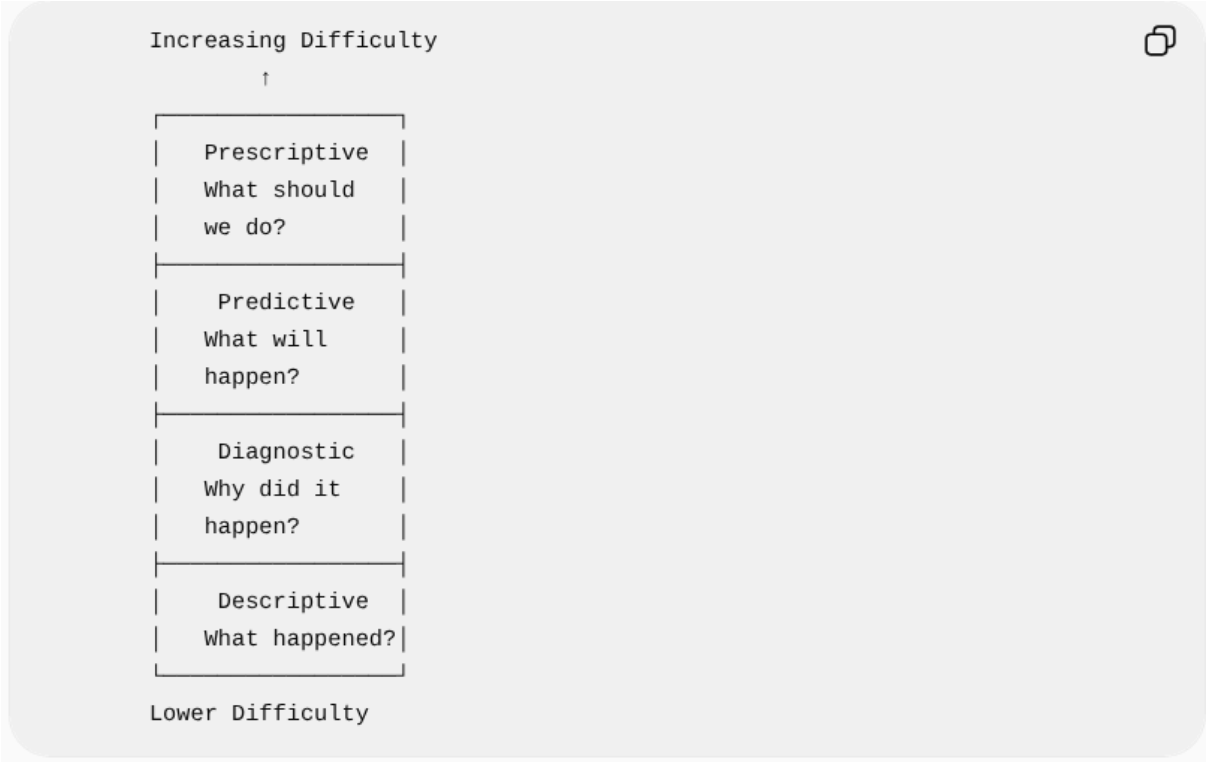
Conclusion

Big Data has applications across almost every major industry. By analysing large and diverse datasets, organisations can **make better decisions, predict future trends, reduce costs, improve customer experiences, and optimise their operations**. Thus, Big Data Analytics has become an important tool for modern businesses and public services.

3. Explain the various categories of data analytics and illustrate how they progress in terms of difficulty.

Data Analytics is the process of discovering meaningful patterns and information from data. It helps answer questions such as **what happened, why it happened, what will happen, and what action should be taken**.

The major categories of analytics can be arranged in increasing order of **difficulty and complexity** as follows:



1. Descriptive Analytics

Descriptive analytics describes and summarizes **past events**. It answers the question:

> **"What happened?"**

It uses historical data to generate reports, summaries, dashboards, and statistics.

Example:
A company analyses last year's sales data and finds that **sales increased by 20% in December**.

Difficulty: Lowest, because it mainly summarizes existing historical data.

---

2. Diagnostic Analytics

Diagnostic analytics goes one step further and tries to identify **why something happened**.

It answers:

> **"Why did it happen?"**

It compares different datasets and looks for relationships, patterns, and possible causes.

Example:
A company notices that December sales increased and analyses customer and sales data to determine that the increase was mainly due to a **festival discount campaign**.

Difficulty: Higher than descriptive analytics because it requires finding relationships and causes.

---

3. Predictive Analytics

Predictive analytics uses **historical data, statistical methods, and machine learning techniques** to predict future outcomes.

It answers:

> **"What will happen?"**

The prediction is based on patterns found in previous data.

Example:
A retailer analyses previous sales, customer behaviour, and seasonal trends to predict that **sales will increase next month**.

Predictive modelling uses data mining and probability to forecast outcomes. It includes techniques such as **classification and regression**.

Difficulty: Higher because it requires prediction rather than simply explaining existing data.

---

4. Prescriptive Analytics

Prescriptive analytics is the most advanced category. It not only predicts what may happen but also recommends **what action should be taken**.

It answers:

> **"What should we do?"**

It combines descriptive and predictive analysis with recommendations to take advantage of future situations. The BDA material describes this as Analytics 3.0.

Example:
If analytics predicts that demand for a product will increase next month, prescriptive analytics may recommend:

"Increase inventory by 20% and offer additional delivery capacity."

Difficulty: Highest because it requires prediction as well as decision-making and recommendation.

Comparison

Category	Main Question	Purpose	Example
Descriptive	What happened?	Summarise past data	Last month's sales
Diagnostic	Why did it happen?	Find causes	Why sales increased
Predictive	What will happen?	Forecast future	Predict next month's sales
Prescriptive	What should we do?	Recommend actions	Increase inventory

How Difficulty Progresses

```
Descriptive
     ↓
What happened?
     ↓
Diagnostic
     ↓
Why did it happen?
     ↓
Predictive
     ↓
What will happen?
     ↓
Prescriptive
     ↓
What should we do?
```

Thus, the complexity increases because analytics moves from **simply understanding the past → finding causes → predicting the future → recommending actions**.

Conclusion

The four major categories of data analytics are **Descriptive, Diagnostic, Predictive, and Prescriptive Analytics**. They progress from relatively simple analysis of past events to complex prediction and decision-making. In short:

Descriptive → Diagnostic → Predictive → Prescriptive

This progression represents an increasing level of **difficulty, intelligence, and decision-making capability**.